

Banco de preguntas — Defensa del TFM

Desarrollo de una aplicación de gestión de planes de boda Laura Rosón Lavín · Tutor: Juan José Ramos
Muñoz Máster en Ingeniería Informática · Universidad de Granada

Cómo usar este documento

- ★★ ★ = muy probable · ★★ = probable · ★ = posible
- **No memorices las respuestas.** Están escritas en un español natural para que entiendas el razonamiento; en la defensa dilo **con tus palabras**. Una respuesta recitada se nota.
- Las respuestas van en el bloque citado (>). Lo que está fuera es contexto para ti.
- Si una respuesta te parece larga, quédate con la **primera frase**: suele bastar. El tribunal repreguntará si quiere más.

Las 12 cifras que debes saber de memoria

Dato	Valor
Invitados de media por boda (sector)	117 · 13 profesionales
Objetivos específicos	8, todos alcanzados
Pruebas automatizadas	18, en 5 ficheros, todas en verde
Conjunto de datos de validación	64 invitados (50 principales + 14 acompañantes)
...más	6 grupos, 9 mesas, 18 tareas, 7 eventos, 11 proveedores, 12 partidas
Dedicación estimada	600 horas
Coste total estimado	13.297 € (13.125 € personal)
Coste real de la IA	~5 € (pago por consumo)
Modelo de IA	gpt-4o-mini, temperatura 0,4
Asistencias de IA	3: tareas, mesas, presupuesto
Duración del proyecto	~34 semanas (nov–jun)
Limitación principal	Sin pruebas automatizadas de frontend

Bloque 1 · Apertura y visión general

★★ ★ Resuma brevemente su trabajo.

Organizar una boda implica coordinar durante meses invitados, mesas, tareas, presupuesto y proveedores, y todo está relacionado entre sí. He desarrollado una aplicación web que centraliza toda esa gestión en un único panel y que incorpora tres asistencias de inteligencia artificial: proponen el listado de tareas, la distribución de los invitados en las mesas y el reparto del presupuesto. La clave de diseño es que la IA nunca decide: propone, el sistema verifica que la propuesta sea válida, y la pareja confirma antes de que se guarde nada.

☆☆☆ ¿Cuál es la aportación de su trabajo? ¿Qué tiene de novedoso?

La novedad no es usar IA, eso hoy lo hace cualquiera. Es **dónde** y **cómo** se aplica. Las plataformas existentes usan la IA para su modelo de negocio —recomendar proveedores— o para redactar textos. Yo la aplico a las decisiones organizativas internas de la pareja, que son las que de verdad dan trabajo. Y lo hago con un patrón concreto y reutilizable: salida con formato estricto, validación contra ese formato, propuesta editable y confirmación explícita del usuario. Ese patrón es la aportación, más que la aplicación en sí.

☆☆ ¿Por qué eligió este tema?

Por dos motivos. Uno objetivo: es un dominio con una necesidad real, muy estructurado y por tanto muy adecuado para sistematizarlo con software, y con un hueco claro que detecté al analizar las plataformas existentes. Y otro personal: la organización de una boda es para mí un objetivo cercano, y me interesaba dedicar el trabajo a una herramienta con utilidad real y no a un ejercicio teórico.

☆☆ Si tuviera que explicar el proyecto a alguien no técnico, ¿qué le diría?

Que es una especie de «centro de mando» para organizar una boda: en un mismo sitio tienes la lista de invitados, quién se sienta en cada mesa, las tareas pendientes, el dinero y los proveedores. Y que además tiene un ayudante que te propone borradores —el listado de tareas, o una distribución de mesas— para que no partas de cero, pero que tú siempre revisas y decides.

☆ ¿A quién va dirigida la aplicación?

A la pareja que organiza su boda, que es un usuario final no técnico. Por eso el producto está íntegramente en español, con formatos de fecha y moneda españoles, y por eso cuidé que las acciones destructivas pidan confirmación y que cada acción dé una respuesta visible.

Bloque 2 · Problema y estado del arte

☆☆☆ ¿Qué aporta su aplicación frente a Bodas.net, The Knot o Zola?

Dos cosas. Primera, el modelo: esas plataformas son *marketplaces*; sus ingresos vienen del directorio de proveedores y las listas de regalos, y las herramientas de organización son un complemento de ese negocio. Algunas funciones concretas, como el plano de mesas de Zola, son incluso de pago. La mía es autocontenida, sin capa comercial. Y segunda, la IA: la suya sirve a su negocio, la mía a las decisiones internas de la pareja.

★ ★ ¿Por qué no usar simplemente una de esas plataformas, que ya están hechas?

Como usuaria, es una opción razonable. Como proyecto, el interés está en el hueco detectado: ninguna aplica la IA a automatizar las decisiones organizativas, y quería explorar si eso se podía hacer de forma fiable, que es un problema técnico no trivial, porque implica que un modelo de lenguaje produzca datos estructurados en los que puedas confiar lo bastante como para guardarlos.

★ ★ ¿Cómo hizo el análisis del estado del arte?

Analicé tres plataformas representativas: Bodas.net como referente español, y The Knot y Zola como líderes internacionales. Revisé sus herramientas de organización funcionalidad a funcionalidad — invitados, mesas, presupuesto, tareas, RSVP—, su modelo de negocio y el uso que hacen de la IA, y lo recogí en una tabla comparativa frente a mi propuesta. Lo complementé con datos de mercado del sector.

★ ¿Qué tamaño tiene este mercado?

El software de planificación de bodas se estimó en torno a 1.500 millones de dólares en 2024, con previsión de crecimiento sostenido. Y el dato que uso como gancho: una boda implica de media unos 117 invitados y la contratación de unos 13 profesionales distintos.

★ ¿Se planteó que fuera un producto comercial?

No, el alcance es académico y por eso es autocontenido y sin capa comercial. Dicho esto, si se planteara, el trabajo por hacer no sería tanto funcional como de cumplimiento: protección de datos, condiciones de servicio y un despliegue con garantías.

Bloque 3 · Objetivos, alcance y metodología

★ ★ ¿Ha cumplido todos los objetivos?

Los ocho objetivos específicos están alcanzados, y con ellos el general. Hay un matiz honesto en el octavo, el de calidad: se ha atendido la seguridad, la usabilidad, la robustez, las pruebas del backend y la integración continua, pero el frontend no tiene todavía pruebas automatizadas propias. Lo recojo explícitamente como limitación y como primera línea de trabajo futuro.

★ ★ ¿Qué metodología siguió y por qué?

Un desarrollo **iterativo e incremental**, en cuatro iteraciones. Al trabajar sola y sin un cliente externo, no tenía sentido montar la ceremonia completa de una metodología ágil de equipo. Lo que sí adopté es su esencia: en cada iteración se analiza, se diseña, se implementa y se prueba un conjunto de funcionalidades, y al final de cada una hay algo que funciona. Eso permitió corregir el rumbo, y de hecho varias mejoras del producto final salieron de probar lo construido, no de la planificación inicial.

★ ★ ¿Cómo organizó el trabajo en el tiempo?

Unas 34 semanas, de noviembre a junio. Empecé con cuatro semanas de análisis y diseño inicial, luego cuatro iteraciones solapadas: el núcleo —autenticación e invitados—, la organización —grupos, mesas, tareas y agenda—, los módulos económicos, y por último las funcionalidades avanzadas y la IA. Las pruebas y la redacción de la memoria se solaparon con las últimas iteraciones.

★ ¿Cuánto ha costado el proyecto?

Unas 600 horas de desarrollo. Aplicando tarifas de referencia de mercado, el coste de personal sería de unos 13.125 €; sumando amortización de hardware y software, unos 13.297 €. Las herramientas son todas de código abierto o con plan gratuito, así que el único coste real de software fueron unos 5 € de consumo de la API de IA.

★ ¿Hubo desviaciones respecto a la planificación?

La principal fue que la distribución de mesas con IA me llevó bastante más de lo previsto, por el problema de los identificadores dinámicos. Y las pruebas del frontend, que estaban previstas en la última iteración, quedaron fuera por falta de tiempo; preferí reconocerlo como limitación antes que añadir pruebas superficiales.

★ ¿Qué dejó fuera del alcance deliberadamente?

Un asistente conversacional tipo chatbot: lo descarté por decisión de producto, porque la interacción por propuestas estructuradas encaja mejor con el flujo de trabajo. También el despliegue en producción, la aplicación móvil nativa y el multi-idioma.

Bloque 4 · Arquitectura

★ ★ ★ Explique la arquitectura del sistema.

Es una arquitectura cliente-servidor de tres niveles. El cliente es una aplicación de página única en React que se ejecuta en el navegador. El servidor es una API REST en Node.js con Express. Y la persistencia es una base de datos relacional PostgreSQL, accedida mediante el ORM Prisma. El código vive en un monorepo con dos paquetes independientes que se ejecutan y despliegan por separado.

☆☆☆ ¿Por qué separa controladores y servicios?

Para que cada capa tenga una única responsabilidad. El controlador se ocupa de **toda** la validación de la entrada y de dar forma a la respuesta HTTP; el servicio contiene la lógica de negocio y el acceso a datos, y no sabe nada de HTTP. Eso tiene dos ventajas prácticas: el servicio se puede probar sin levantar un servidor, y si mañana quisiera exponer la misma lógica por otro canal, reutilizaría los servicios sin tocarlos.

☆☆ ¿Por qué un monorepo y no dos repositorios?

Porque al desarrollar sola, tener cliente y servidor en el mismo repositorio simplifica mucho mantener sincronizados los cambios que afectan a ambos, que son constantes: si cambio un campo en la API, toco el mismo *commit* en los dos lados. Los paquetes siguen siendo independientes, con sus propias dependencias y scripts.

☆☆ ¿Qué es esa «capa única de acceso a la API» del cliente?

Es un módulo por el que pasan **todas** las llamadas de red del frontend. Centraliza tres cosas transversales: inyectar el token de sesión en cada petición, renovar la sesión de forma transparente cuando el token caduca, y extraer el mensaje de error del cuerpo de la respuesta para que los componentes muestren siempre un texto limpio. Al estar en un solo sitio, el comportamiento es idéntico en toda la aplicación y no se repite en cada pantalla.

☆ ¿Consideró una arquitectura de microservicios?

No, y creo que habría sido un error. Para un dominio de este tamaño y un solo desarrollador, los microservicios añaden complejidad operativa —despliegue, comunicación, consistencia— sin aportar nada. El monolito bien estructurado en capas por dominio da la separación que necesito y, si algún día hiciera falta, esos módulos son la línea natural por la que partirlo.

☆ ¿Por qué Express y no NestJS o Fastify?

Por madurez y simplicidad. NestJS impone una estructura muy completa con decoradores e inyección de dependencias, que para un proyecto de este tamaño es andamiaje excesivo. Fastify es más rápido, pero la diferencia de rendimiento es irrelevante aquí y su ecosistema es menor. Express tiene el catálogo de *middlewares* más amplio y una curva de entrada suave.

Bloque 5 · Modelo de datos

☆☆☆ ¿Por qué una base de datos relacional y no NoSQL?

Porque el dominio es intrínsecamente relacional: un invitado pertenece a una boda, a un grupo y a una mesa; una partida de presupuesto se vincula a un proveedor. Necesito integridad referencial y consultas que cruzan entidades. Una base documental me obligaría a duplicar datos o a resolver las relaciones en el código, que es justo lo que quiero evitar.

★ ★ ★ ¿Cómo evita que dos invitados ocupen el mismo asiento?

Con una **restricción de unicidad** sobre la pareja mesa-asiento, impuesta en la propia base de datos. Es una decisión deliberada: no es una comprobación en el código, que podría saltarse si añado una vía nueva. Da igual si la petición viene de la interfaz, de la API o de la IA: la base de datos rechaza el duplicado. La garantía está en el sitio donde no se puede eludir.

★ ★ ¿Cómo modela los acompañantes?

Con una **relación reflexiva**: un acompañante es un invitado que apunta a su invitado principal, mediante un campo que referencia a la propia tabla. Así evito duplicar una tabla casi idéntica y un acompañante puede tener los mismos atributos —dieta, alergias, confirmación—. El borrado del principal arrastra en cascada a sus acompañantes.

★ ★ ¿Qué pasa si borro una boda?

Se propaga en cascada a todas sus entidades hijas: invitados, grupos, mesas, tareas, eventos, presupuesto y proveedores. El sistema queda consistente, sin registros huérfanos. Está declarado en el esquema, no resuelto a mano en el código.

★ ¿Y si borro un proveedor que está vinculado a una partida de presupuesto?

Ahí la política es distinta: no se borra la partida, se **desvincula** automáticamente. La partida conserva además un campo de proveedor en texto libre como respaldo, así que no se pierde la información de quién era.

★ ¿Está normalizado el modelo? ¿Ha desnormalizado algo?

Está normalizado en lo esencial. La excepción consciente es ese campo de proveedor en texto libre que convive con la relación al proveedor: es una redundancia deliberada para no perder el dato si el proveedor se elimina o si aún no está dado de alta.

★ ¿Ha puesto índices?

Sí, sobre las claves foráneas por las que se filtra constantemente: la boda, el grupo, la mesa y el invitado principal. Como prácticamente toda consulta se acota por boda, ese índice es el más relevante.

Bloque 6 · API REST

★ ★ ¿Cómo ha diseñado la API?

Como una API REST que intercambia JSON sobre HTTP. Los recursos se organizan por dominio bajo el prefijo `/api`, con los verbos HTTP usados de forma convencional: GET para leer, POST para crear, PUT o PATCH para modificar y DELETE para borrar. Salvo las rutas públicas —salud, autenticación y confirmación de asistencia—, todas exigen un token en la cabecera `Authorization: Bearer`.

★ ★ ¿Qué códigos de respuesta usa y cuándo?

200 para éxito; 400 cuando falla la validación de entrada, con mensaje legible en español; 401 si falta el token o ha caducado; 404 si el recurso no existe **o no pertenece al usuario**; 409 en conflictos, como registrarse con un email ya existente; 500 genérico para errores inesperados, sin filtrar detalles internos; y 502 cuando falla el servicio externo de IA.

★ ¿Por qué 404 y no 403 al acceder a una boda ajena?

Porque un 403 significa «existe pero no puedes», y eso ya es información que estoy filtrando. Con un 404 el atacante no puede distinguir entre una boda que no existe y una que existe pero no es suya. Es una decisión de seguridad deliberada y está verificada con una prueba.

★ ¿Ha versionado la API?

No, y es una simplificación consciente: al ser el único consumidor mi propio frontend, puedo cambiar ambos a la vez. En cuanto hubiera clientes externos —una app móvil, por ejemplo— el versionado sería necesario, y la forma natural sería un prefijo de versión en la ruta.

★ ¿Hay paginación?

No la implementé, porque los volúmenes de una boda son pequeños por naturaleza: unos cientos de invitados como mucho. Es una simplificación que reconozco; si el dominio creciera, la paginación en los listados sería lo primero que añadiría.

Bloque 7 · Seguridad ★ *Bloque muy probable*

★ ★ ★ ¿Cómo garantiza que un usuario no acceda a los datos de otro?

Es el riesgo clásico de referencia directa insegura a objetos, el IDOR. Lo ataco en dos frentes. Primero, un **middleware de propiedad** que intercepta las peticiones con identificador de boda y comprueba que esa boda pertenece al usuario del token; si no, responde 404. Y segundo, las rutas que operan por identificador filtran además por propietario en la propia consulta a la base de datos, así que aunque alguien se saltara el middleware, la consulta no devolvería nada. Hay una prueba automatizada que verifica el acceso cruzado.

☆☆☆ ¿Cómo funciona la autenticación?

Con tokens JWT y un esquema de dos tokens. Al hacer login el servidor entrega un **token de acceso**, de vida corta, que viaja en cada petición, y un **token de refresco**, de vida más larga, que solo sirve para pedir un acceso nuevo. La ventaja es que si el de acceso se ve comprometido caduca pronto, y el de refresco se guarda en base de datos, así que se puede revocar de verdad: eso es lo que hace el cierre de sesión.

☆☆ ¿Qué pasa si el token caduca mientras el usuario está trabajando?

No se le expulsa. Cuando una petición falla con 401 por caducidad, la capa de API del cliente pide un token nuevo con el de refresco y **reintenta la petición una sola vez**, de forma transparente. Además, si hay varias peticiones en vuelo a la vez, comparten una única renovación en lugar de lanzar varias. Solo si el refresco también ha caducado se redirige al login.

☆☆ ¿Cómo almacena las contraseñas?

Cifradas con **bcrypt**, que aplica un hash con sal. Nunca se guarda la contraseña en claro y el hash no es reversible. Hay una prueba unitaria que comprueba que el cifrado y la verificación posterior funcionan y distinguen una contraseña incorrecta.

☆☆ ¿Está protegido contra ataques de fuerza bruta?

Sí, hay limitación de frecuencia sobre las rutas de autenticación: una ventana de 15 minutos con un máximo de intentos configurable, que responde 429 al excederse. Está acotado a `/api/auth/*`, que es donde tiene sentido.

☆☆ ¿Y la inyección SQL?

No construyo SQL concatenando cadenas en ningún punto: todo el acceso a datos pasa por el ORM, que parametriza las consultas. Eso elimina el vector por construcción.

☆ ¿Y XSS?

En el cliente, React escapa por defecto el contenido que se renderiza, así que no inyecto HTML crudo. Donde sí tuve que ser explícita fue en la **plantilla de correo** de las invitaciones, porque ahí sí compongo HTML: escapo los datos del invitado antes de insertarlos, y hay una prueba unitaria que lo verifica.

★ ¿Cómo gestiona los secretos?

Todos en variables de entorno, nunca en el código. Y hay una validación al arrancar: en producción, si faltan los secretos de firma de los tokens o tienen todavía el valor por defecto, **el servidor no arranca**. En desarrollo solo avisa, para no bloquear el trabajo local.

★ ¿Y CORS?

Abierto en desarrollo, y en producción restringido a una lista de orígenes permitidos definida por configuración.

★★ ¿Ha tenido en cuenta la protección de datos? Maneja datos personales.

(Reconoce el punto: es una pregunta legítima y bien planteada.) Es una consideración real y una limitación del trabajo. Se almacenan nombres, emails, teléfonos y alergias, y las alergias son datos de salud, que tienen una protección especial. Técnicamente están aislados por boda y las contraseñas cifradas, pero un despliegue real exigiría cumplimiento formal del RGPD: base legal para el tratamiento, información al interesado, política de privacidad, derecho de supresión y una política de retención. No estaba en el alcance del TFM y lo asumo como una carencia.

★ La página pública de confirmación no pide login. ¿No es un riesgo?

Es un compromiso consciente entre seguridad y usabilidad: no puedes pedirle a un invitado que se cree una cuenta para decir si va a la boda. El acceso va por un **token único** por invitado, largo y no adivinable, que da acceso solo a sus propios datos de asistencia y a los de sus acompañantes. Quien no tenga el enlace no puede llegar. Un refuerzo razonable sería darle caducidad.

Bloque 8 · Inteligencia artificial ★ *El bloque estrella*

★★★ Explique cómo ha integrado la IA.

Con un flujo de cinco pasos común a las tres asistencias. El servidor construye el contexto con datos reales de la boda; llama al modelo **obligándole a responder con un esquema JSON estricto**; valida la respuesta contra ese mismo esquema; devuelve una **propuesta editable**; y solo cuando el usuario confirma se persiste, pasando por los mismos puntos de alta que usa el resto de la aplicación. La IA nunca escribe en la base de datos.

★★★ ¿Qué pasa si el modelo devuelve algo que no esperaba? ¿Y las alucinaciones?

Hay tres barreras. La primera es preventiva: los *structured outputs* con `strict: true` obligan a que la respuesta se ajuste al esquema. La segunda es de verificación: aunque venga garantizada, la valido igualmente con Zod, por defensa en profundidad; no delego mi integridad en un servicio externo. Y la tercera, para el caso de las mesas, es semántica: comparo la propuesta con el estado real de la base de datos. Si algo falla, se descarta y el usuario recibe un mensaje claro. El peor caso posible es que no haya propuesta, nunca que se guarde basura.

★ ★ ★ ¿Por qué no deja que la IA aplique los cambios directamente?

Por una razón de diseño y otra técnica. La de diseño: organizar una boda tiene un componente personal que un modelo no puede conocer —qué familiares no se hablan, a quién quieres cerca—. La técnica: si la IA escribiera directamente, un fallo del modelo corrompería datos reales. Como propuesta editable, el coste de un error es que el usuario desmarque una sugerencia.

★ ★ ★ ¿Qué son los *structured outputs* y por qué son importantes aquí?

Un modelo de lenguaje devuelve texto, y el texto no es fiable como dato: puede traer una coma de más, añadir explicaciones o cambiar el nombre de un campo. Los *structured outputs* son un mecanismo por el que le pasas al proveedor un esquema JSON y él **garantiza** que la respuesta se ajusta a ese esquema. Eso convierte la salida del modelo en algo que un programa puede consumir con confianza, que es exactamente lo que necesito para poder guardarlo.

★ ★ La distribución de mesas fue el caso difícil. ¿Por qué?

Porque los identificadores de mesas e invitados son **dinámicos**: se generan en cada boda, así que no puedo enumerarlos por adelantado en el esquema. El esquema estricto garantiza la *forma* de la respuesta, pero no que los identificadores existan. La IA podía devolverme el identificador de una mesa inexistente, un asiento fuera de la capacidad o repetir a una persona. Tuve que añadir una capa que valida y depura la propuesta contra el estado real antes de mostrarla, y aplicarla dentro de una transacción que respeta la restricción de unicidad.

★ ★ ¿Qué modelo usa y por qué ese?

`gpt-4o-mini`. Es un modelo pequeño, mucho más barato y rápido que los grandes, y para esta tarea —generar listas estructuradas a partir de un contexto acotado— es más que suficiente. No necesito razonamiento complejo, necesito fiabilidad de formato y coherencia, y eso lo da. El modelo además es configurable por variable de entorno, así que cambiarlo no requiere tocar código.

★ ★ ¿Qué parámetros usa en las llamadas?

Temperatura 0,4, que es baja: quiero respuestas consistentes, no creativas. Un límite de tokens de salida acotado, para controlar el coste. Y un tiempo de espera y un número máximo de reintentos en el cliente, para que un fallo del proveedor no deje la petición colgada.

★ ★ ¿Cuánto cuesta operar esto?

Es pago por consumo, no suscripción: aproximadamente 0,15 \$ por millón de tokens de entrada y 0,60 \$ por millón de salida. Durante todo el desarrollo, con muchas pruebas de las tres asistencias, el coste total fue de unos 5 €.

★ ★ ¿Ha evaluado la calidad de las propuestas de la IA?

(Respuesta honesta, no la inflés.) No de forma cuantitativa. La validación fue funcional: comprobé con generaciones reales que las propuestas son estructuralmente válidas y coherentes con el contexto — por ejemplo, que las tareas se retro-planifican desde la fecha de la boda y no repiten las ya existentes —. Una evaluación sistemática, comparando por ejemplo la distribución de mesas propuesta con una hecha por una persona, o midiendo qué porcentaje de propuestas acepta el usuario sin editar, sería una línea de trabajo futuro clara.

★ ★ ¿Qué pasa si el servicio de IA no está disponible?

La aplicación **degrada con elegancia**. Si no hay clave configurada, el frontend consulta un endpoint de estado y directamente no muestra los botones de IA; el resto funciona igual. Si la clave existe pero la llamada falla, se devuelve un error controlado con un mensaje amable, sin filtrar detalles internos. En ningún caso se bloquea el uso normal de la aplicación.

★ ¿Envía datos personales de los invitados a OpenAI?

(Pregunta muy pertinente, reconócelo.) Sí, en el caso de las mesas se envían nombres y grupos, que son datos personales. Es una consideración de privacidad real. Se envía el mínimo necesario —no van emails ni teléfonos— y las llamadas salen solo del servidor, nunca del cliente. Pero en un despliegue real habría que informarlo expresamente en la política de privacidad y, idealmente, seudonimizar: mandar identificadores y etiquetas de grupo en vez de nombres reales. Es una mejora que asumo.

★ ¿Podría usar un modelo local en vez de un servicio externo?

Sí, y sería una evolución interesante, porque resolvería de golpe la dependencia externa, el coste y la privacidad. La arquitectura lo permite: toda la interacción con el modelo está encapsulada en una única función, así que el cambio estaría acotado a ese punto. La duda sería si un modelo que pueda correr en local mantiene la fiabilidad de los *structured outputs*.

★ ¿Cómo construye los *prompts*?

Cada asistencia aporta dos cosas: un mensaje de sistema que fija el rol y las reglas, y un mensaje de usuario que se construye programáticamente con el contexto de la boda leído de la base de datos: fecha, número de invitados, elementos ya existentes para no duplicarlos. No hay texto que escriba el usuario final directamente en el *prompt*, salvo unas notas opcionales de estilo en el asistente de presupuesto.

Bloque 9 · Frontend

★ ★ ¿Por qué React y no Angular o Vue?

Por el modelo de componentes, que es simple y encaja con una interfaz modular como esta, y por el tamaño del ecosistema: encontré bibliotecas maduras para el calendario y las gráficas. Angular es más completo pero mucho más prescriptivo y pesado para este tamaño de proyecto. Vue habría sido una alternativa perfectamente válida; fue una decisión de familiaridad y ecosistema.

★ ¿Cómo gestiona el estado? ¿Usó Redux?

No hizo falta. El estado de la aplicación es mayoritariamente **estado de servidor**: datos que se piden, se muestran y se vuelven a pedir. Eso lo resuelve cada página con el estado propio de React y los servicios de dominio. Lo único verdaderamente global —los tokens y la boda activa— vive en el almacenamiento local del navegador. Meter Redux habría sido complejidad sin beneficio.

★ ¿Por qué guarda el token en `localStorage` y no en una cookie?

Es un compromiso. `localStorage` es sencillo y funciona bien con una API consumida por una SPA. Su desventaja es que es accesible por JavaScript, así que en caso de XSS sería vulnerable. La alternativa más segura sería una cookie `httpOnly`, que obliga a manejar protección CSRF. Lo reconozco como una decisión mejorable en un despliegue real.

★ ★ ¿Es accesible la aplicación?

Hice una auditoría específica. Añadí etiquetas accesibles a los botones que son solo icono, la asociación explícita entre etiqueta y campo en todos los formularios de autenticación, y revisé el comportamiento en móvil: la barra lateral se oculta y el menú pasa a un panel deslizante que antes no funcionaba. Queda pendiente asociar etiquetas en algunos campos de diálogos concretos.

★ ¿Es responsive?

Sí, se diseñó con ese requisito. La navegación cambia en móvil, las tablas anchas tienen desplazamiento horizontal propio, y la interfaz funciona tanto en modo claro como oscuro.

Bloque 10 · Pruebas y calidad

★ ★ ★ ¿Cómo ha probado el sistema?

En tres niveles. **Pruebas unitarias** sobre las utilidades con lógica propia, sin base de datos: cifrado de contraseñas, tokens, la plantilla de invitación y el manejador de errores. **Pruebas de integración** que hacen peticiones HTTP reales contra una base de datos de verdad y recorren el flujo de registro, boda e invitado, incluyendo los casos de error. Y **verificación funcional manual** módulo a módulo sobre la interfaz. Todo ello respaldado por integración continua.

★ ★ ¿Cuántas pruebas tiene y qué cubren?

18 pruebas automatizadas repartidas en 5 ficheros, todas en verde. Cubren la autenticación, el alta y listado de invitados, el aislamiento entre bodas, y el tratamiento uniforme de errores.

★ ★ ★ ¿Por qué no hay pruebas de frontend?

(Con naturalidad, sin excusarte de más.) Es la principal limitación del trabajo. Prioricé cubrir el backend, que es donde está la lógica de negocio y las garantías de seguridad; con tiempo limitado, me pareció que era donde una prueba automatizada aporta más. El frontend queda validado por la compilación con verificación de tipos, que detecta una clase importante de errores, y por pruebas funcionales manuales. Lo recojo explícitamente como primera línea de trabajo futuro.

★ ¿Qué cobertura de código tiene?

No la medí como métrica formal. Hablar de porcentaje de cobertura teniendo el frontend sin pruebas habría dado una cifra engañosa, así que preferí describir qué se cubre y qué no.

★ ★ ¿Qué es la integración continua que menciona?

Un flujo en GitHub Actions que se ejecuta con cada cambio. Tiene dos trabajos: el de backend levanta una base de datos PostgreSQL de servicio, instala dependencias, genera el cliente de datos, pasa el análisis de estilo, compila, aplica las migraciones y lanza las pruebas; el de frontend instala, analiza y compila. Que ambos estén en verde es condición para integrar. Actúa como red de seguridad frente a regresiones.

★ ¿Qué es una regresión?

Romper sin querer algo que antes funcionaba, al hacer un cambio en otra parte. Es justo el riesgo que crece cuando un proyecto se hace grande, y la razón de tener pruebas automáticas ejecutándose solas en cada cambio.

★ ¿Usa análisis estático?

Sí, ESLint con la configuración de TypeScript en ambos paquetes, y forma parte del flujo de integración continua. El backend está sin errores; el frontend tiene cero errores y algunos avisos de estilo que dejé conscientemente como avisos para no bloquear el flujo.

Bloque 11 · Despliegue, operación y escalabilidad

★ ★ ¿Está desplegada? ¿Cómo se desplegaría?

Funciona en entorno local y está preparada para desplegarse, pero el despliegue en producción no entraba en el alcance. Haría falta un servidor para la API, una base de datos PostgreSQL gestionada, servir el frontend ya compilado como estáticos, y configurar las variables de entorno, incluido un servidor SMTP real. La base de datos ya se ejecuta en contenedor Docker en desarrollo, lo que facilita reproducir el entorno.

★ ★ ¿Esto escala? ¿Y si tuviera 10.000 usuarios?

El diseño ayuda en lo fundamental: la autenticación es sin estado, así que se pueden levantar varias instancias del servidor detrás de un balanceador sin sesiones compartidas. Los tres puntos que revisaría son: los índices y la paginación en los listados; los documentos adjuntos, que ahora se guardan en disco local y habría que mover a un almacenamiento de objetos; y las llamadas a la IA, que al ser lentas y de pago convendría encolar en vez de hacerlas síncronas.

★ ¿Hay copias de seguridad? ¿Monitorización?

No, y es una carencia de operación que reconozco. Son requisitos de un despliegue real, no del prototipo académico: copias periódicas de la base de datos, registro centralizado y alertas.

★ ¿Cómo envía los correos?

Con Nodemailer sobre SMTP. El transportador se inicializa de forma perezosa: si hay credenciales SMTP configuradas las usa, y si no, cae a una **cuenta de prueba** que no entrega el correo a destinatarios reales pero devuelve un enlace de previsualización. Eso me permitió desarrollar y probar todo el flujo de invitaciones sin montar infraestructura de correo.

★ ¿Dónde se guardan los documentos de los proveedores?

En disco local del servidor, con nombre generado aleatoriamente para evitar colisiones y adivinación, validando tipo y tamaño —máximo 10 MB, solo PDF, imágenes y ofimática—. La descarga pasa por un endpoint autenticado que verifica la propiedad, no es una URL pública. Borrar un proveedor limpia también sus ficheros del disco.

Bloque 12 · Dificultades y visión crítica

★ ★ ★ ¿Qué ha sido lo más difícil del proyecto?

Técnicamente, la distribución de mesas con IA, por el problema de los identificadores dinámicos que ya he comentado: fue donde entendí que la salida de un modelo hay que verificarla siempre contra el estado real. Y en el terreno del detalle, la importación CSV: al abrir el fichero en Excel en español los acentos se corrompían y todo caía en una sola columna. La causa eran dos cosas combinadas: Excel necesita la marca de orden de bytes para reconocer UTF-8, y en configuración regional española el separador de columnas es el punto y coma, no la coma. Me pareció un buen recordatorio de que los detalles de integración cuestan tanto como la arquitectura.

★ ★ ¿Qué otras dificultades encontró?

La representación visual de las mesas, que iteré varias veces hasta dar con algo claro. El aislamiento entre bodas, que al principio estaba resuelto solo en algunos módulos y tuve que sistematizar con un middleware. Y la expiración de la sesión: al principio, cuando caducaba el token, la aplicación expulsaba al usuario al login, y resolverlo bien —con renovación transparente y compartida entre peticiones concurrentes— llevó su trabajo.

★ ★ ★ Si empezara de nuevo, ¿qué haría distinto?

Dos cosas. Escribiría las pruebas del frontend desde el principio en lugar de dejarlas para el final, porque al final siempre se come el tiempo. Y definiría antes el contrato de las asistencias de IA: la capa común la extraje después de tener la primera funcionando, y habría sido más limpio diseñarla primero y luego implementar las tres encima.

★ ★ ¿Qué limitaciones reconoce en su trabajo?

Cuatro, por orden de importancia: no hay pruebas automatizadas de frontend; hay dependencia de un servicio externo de pago para la IA; el envío de correo en desarrollo usa una cuenta de prueba y un despliegue real requeriría SMTP propio; y no hay cumplimiento formal de protección de datos, que sería imprescindible en producción.

★ ¿Qué deuda técnica tiene el proyecto?

Las pruebas de frontend, la paginación de los listados, el almacenamiento de ficheros en disco local, y algunos avisos de estilo que dejé como avisos para no bloquear el flujo de integración.

Bloque 13 · Preguntas de concepto

Son preguntas cortas de definición. El tribunal a veces las usa para comprobar que dominas el vocabulario que has empleado.

Pregunta	Respuesta breve
¿Qué es un ORM?	Una capa que traduce entre las tablas de la base de datos y los objetos del lenguaje, para no escribir SQL a mano.
¿Qué es una transacción?	Un conjunto de operaciones que se aplican todas o ninguna. Si una falla, se deshacen todas.
¿Qué significa ACID?	Atomicidad, consistencia, aislamiento y durabilidad: las garantías que ofrece una base de datos transaccional.
¿Qué es una API REST?	Un estilo de interfaz donde cada recurso tiene su URL y se opera con los verbos HTTP estándar.
¿Qué es un JWT?	Una credencial firmada que el servidor emite y verifica sin guardar estado de sesión.
¿Qué es un middleware?	Una función que se ejecuta entre la petición y su manejador final, y que puede validarla, transformarla o cortarla.
¿Qué es una SPA?	Una aplicación web que no recarga la página entera al navegar: pide solo datos y repinta lo que cambia.
¿Qué es idempotencia?	Que repetir la misma operación produzca el mismo resultado. GET, PUT y DELETE lo son; POST no.
¿Qué es una migración?	Un cambio versionado del esquema de la base de datos, que se puede aplicar de forma reproducible.
¿Qué es el tipado estático?	Que los tipos se comprueban al compilar, antes de ejecutar, de modo que muchos errores se detectan antes.
¿Qué es IDOR?	Acceder a datos ajenos manipulando un identificador en la petición. Se evita comprobando la propiedad.
¿Qué es una restricción de unicidad?	Una regla de la base de datos que impide que se repita un valor o una combinación de valores.

Bloque 14 · La pregunta sobre herramientas de IA

Tu tutor la incluye **explícitamente** en su guía de presentación, con este consejo literal:

«¿Qué partes se han hecho con Inteligencia Artificial?» → Sé transparente. Responde con total honestidad sobre las herramientas utilizadas.»

Es además la respuesta estratégicamente más segura: si titubeas, lo minimizas y luego se nota, el daño es mucho mayor que el de haberlo dicho desde el principio.

Cómo estructurar la respuesta

1. **Qué herramientas** usaste, con su nombre.

2. **Para qué exactamente** (¿código? ¿redacción? ¿ambos? ¿qué partes?).

3. **Qué es tuyo:** las decisiones, la comprensión del sistema, la verificación de que funciona.

⚠ **Esta es la única respuesta de este documento que no te escribo.** Y es deliberado: solo tú sabes qué usaste y en qué medida, y una respuesta que no se corresponda con la realidad es exactamente lo que no te conviene.

Habla con tu tutor antes de la defensa. Él conoce el criterio de la escuela sobre el uso de asistentes y sobre cómo debe declararse. Llegar alineado con él te quita toda la presión de esta pregunta, y además es quien puede defenderte si surge.

Bloque 15 · Cuando no sepas algo

Pasará, y es normal. Lo que el tribunal valora es la honestidad y la capacidad de razonar, no saberlo todo.

Fórmulas que funcionan

- «*No lo había considerado desde ese ángulo. Lo que sí puedo decirle es que...*» → y reconduces a terreno que dominas.
- «*No lo medí de forma cuantitativa. Mi validación fue funcional, y una evaluación sistemática sería una línea de trabajo futuro.*»
- «*No lo sé con seguridad. Mi intuición sería [X], pero tendría que comprobarlo.*» → Es una respuesta perfectamente válida y honesta.
- «*Tiene razón, es una limitación del trabajo.*»

Si te repiten una crítica o sugerencia

Dales la razón. No discutas ni tenses la situación. Muchas veces no es un ataque: es una sugerencia de mejora, o quieren ver si sabes encajar una crítica. «*Es un buen apunte, lo recojo*» cierra el tema perfectamente.

Si la pregunta es larga o no la entiendes

Pide que te la repitan o reformula tú: «*Si le entiendo bien, me pregunta por...*». Es señal de rigor, no de debilidad.

Prohibido

- ❌ Inventarte una cifra, una tecnología o un resultado. Es lo único que no se perdona.
- ❌ Responder a la defensiva.
- ❌ Rellenar el silencio hablando. Responde y calla.

Repaso final: las 8 respuestas que sí o sí debes dominar

Si el tiempo se te echa encima, asegura estas:

1. Resumen del trabajo en un minuto.
 2. Qué aporta frente a Bodas.net / The Knot / Zola.
 3. La arquitectura en tres niveles.
 4. Cómo garantizas el aislamiento entre bodas.
 5. **El flujo de la IA: contexto → esquema estricto → validación → propuesta → confirmación.**
 6. **Por qué la IA no escribe directamente en la base de datos.**
 7. Cómo evitas que dos invitados ocupen el mismo asiento.
 8. Por qué no hay pruebas de frontend (y que lo reconoces como limitación).
-

Suerte. Respira, ve despacio y recuerda: nadie en esa sala conoce este trabajo mejor que tú.